

Programación 3

PRESENTADO A
Caceres, Manuel Adrian

PRESENTADO POR
Carrizo, Facundo Martín
Sánchez Agüero, Franco
Yañez, Agustín Emanuel

Cursada de Verano 2025 - Equipo 5

<https://github.com/francoosanchez/Programacion3-verano.git>

11/02/2024

Agenda

1

Recursividad

5

Kruskal

2

Divide y Conquista

6

Dijkstra

3

Greedy

7

Programación
Dinámica

4

Prim

8

Floyd

Recursividad

¿Qué es?

Técnica de programación en la que una función se llama a sí misma para resolver problemas más pequeños hasta llegar a un caso base.

Características principales

- Caso Base
- Llamada recursiva

Aplicaciones

- Matemáticas
- Estructuras de Datos
- Ordenamiento y Búsqueda
- IA y Juegos
- Exploración de Archivos



Consigna a resolver (Recursividad)

Sumar los primeros n números enteros, implementando en java un algoritmo que use recursión.

Realice el análisis de recurrencia.



Resolución ejercicio

```
c2_act5.java

public class Actividad5 {
    public static int sumaRecursiva(int n) {
        if (n <= 0) return 0; // Caso base
        return n + sumaRecursiva(n - 1); // Llamada recursiva
    }

    public static void main(String[] args) {
        int n = 3;
        System.out.println("La suma de los primeros " + n + " números es: " + sumaRecursiva(n));
        // Salida esperada: 6
    }
}
```

Caso base
Función recursiva

$O(n)$

Si llamamos sumaRecursiva(3), el flujo es:

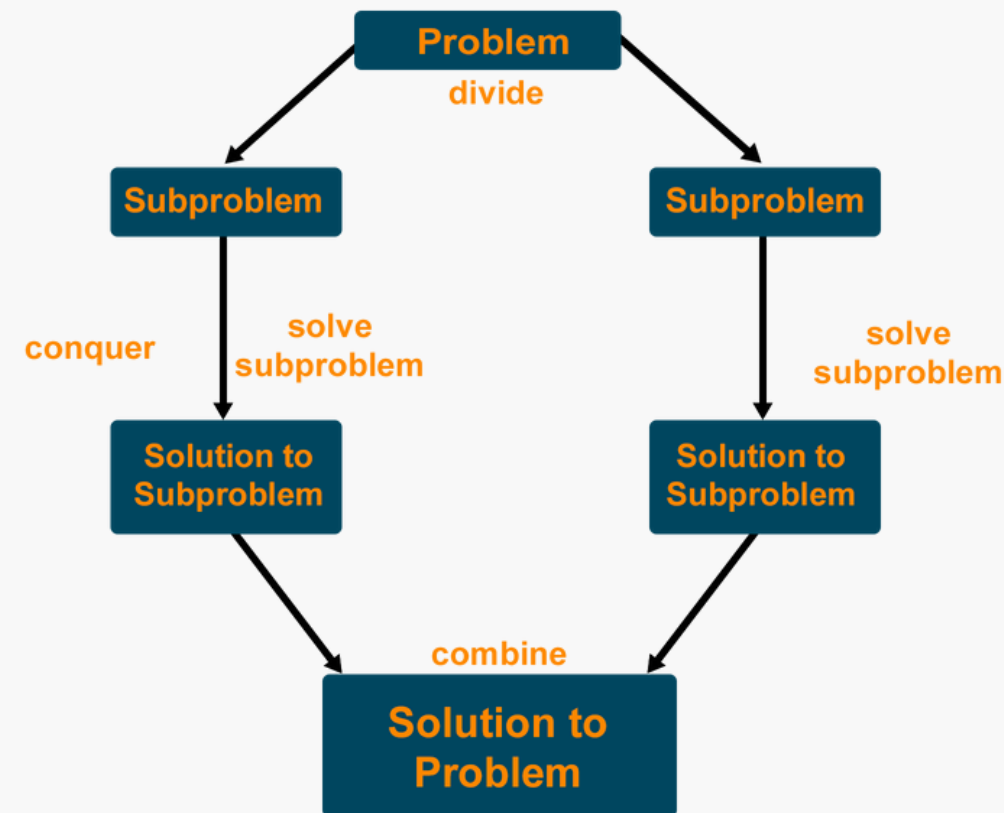
sumaRecursiva(3) -> 3 + sumaRecursiva(2)

sumaRecursiva(2) -> 2 + sumaRecursiva(1)

sumaRecursiva(1) -> 1 + sumaRecursiva(0)

sumaRecursiva(0) -> 0 (caso base)

Divide y Conquista



¿Qué es?

1. Dividir un problema en subproblemas más pequeños
2. Resolver cada subproblema de manera recursiva
3. Combinar las soluciones de los subproblemas para resolver el problema original.

Métodos de ordenamiento

- QuickSort $O(n \log n)$
- MergeSort $O(n \log n)$

Aplicaciones

- Ordenamiento
- Búsqueda
- Álgebra y geometría
- Redes neuronales

Consigna a resolver (D&C)

Aplicar la técnica de Divide y Vencerás en una lista de clientes con id, nombre y scoring, buscando el cliente con el scoring máximo.





c4_act1.java

```
public static Cliente encontrarClienteMaximo(List<Cliente> clientes, int inicio, int fin) {  
    // Caso base: si hay un solo cliente, ese es el máximo  
    if (inicio == fin) {  
        return clientes.get(inicio);  
    }  
  
    // Dividir la lista en dos mitades  
    int medio = (inicio + fin) / 2;  
  
    // Encontrar el máximo en cada mitad  
    Cliente maxIzquierda = encontrarClienteMaximo(clientes, inicio, medio);  
    Cliente maxDerecha = encontrarClienteMaximo(clientes, medio + 1, fin);  
  
    // Retornar el cliente con mayor scoring entre las dos mitades  
    return (maxIzquierda.scoring >= maxDerecha.scoring) ? maxIzquierda : maxDerecha;  
}
```

Caso base

Divido el problema

Máximo de cada
mitad

Combino resultados

Complejidad temporal

1. Caso base: $O(1)O(1)O(1)$.
2. División: $O(1)O(1)O(1)$.
3. Recursión: $T(n)=2T(n/2)T(n)=2T(n/2)T(n)=2T(n/2)$.
4. Combinación: $O(1)O(1)O(1)$.

Entonces, la ecuación de recurrencia es: $2T(n/2)+O(1) = \mathbf{O(n)}$

Greedy

Estrategia

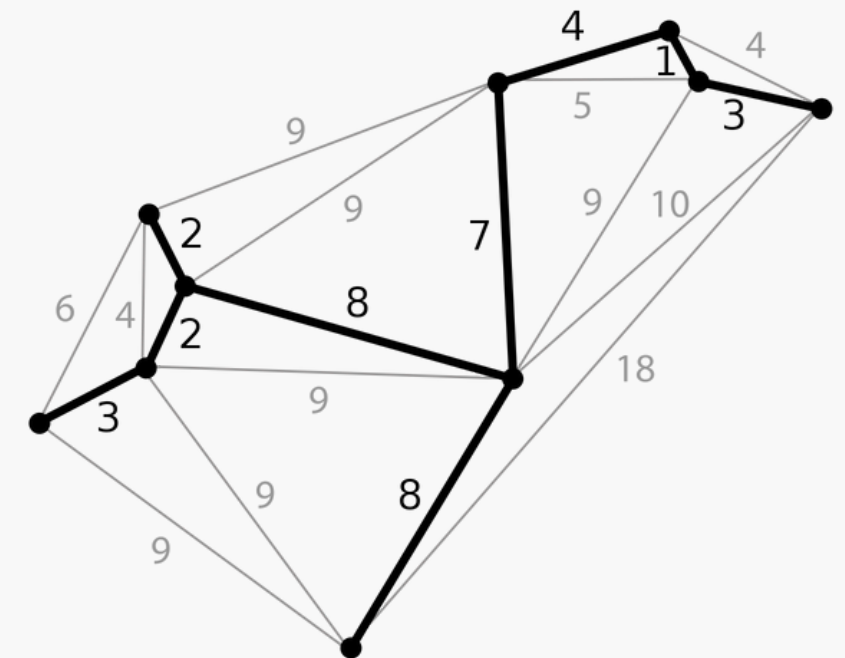
Toma la mejor decisión localmente óptima en cada paso, con la esperanza de encontrar una solución globalmente óptima.

Decisión

Elige la opción óptima y no considera decisiones futuras.

Aplicaciones

Se utiliza en problemas como el cambio de monedas, mochila fraccionaria y árboles de expansión mínima.



Consigna a resolver (Greedy)

Dada una lista de monedas con denominaciones convencionales (10,1,5,2,10,10,5,2,5,5,5,5,5,5,10), implementar una función greedy que devuelva o genere una lista de monedas para dar cambio exacto utilizando una lista de monedas disponible para un importe de \$33. Devolver una lista nula o lanzar una excepción, si no se puede dar el cambio.



Resolución ejercicio

```
c5_act1.java

public class c5_act1 {

    public static List<Integer> darCambio(int[] monedas, int importe) throws Exception {
        // Ordenar las monedas de mayor a menor usando QuickSort
        quickSort.quickSort(monedas, 0, monedas.length - 1);

        List<Integer> cambio = new ArrayList<>();
        for (int moneda : monedas) {
            while (importe >= moneda) {
                cambio.add(moneda);
                importe -= moneda;
            }
        }
        if (importe == 0) {
            return cambio;
        }

        throw new Exception("No se puede dar el cambio exacto");
    }

    public static void main(String[] args) {
        int[] monedas = {10,1,5,2,10,10,5,2,5,5,5,5,5,5,10};
        int importe = 33;

        try {
            List<Integer> resultado = darCambio(monedas, importe);
            System.out.println("Cambio dado: " + resultado);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Llama a la función quickSort

Calcular cambio

Condición de corte

Complejidad temporal

$O(n \log n) + O(n)$

$O(n \log n)$

Resultado

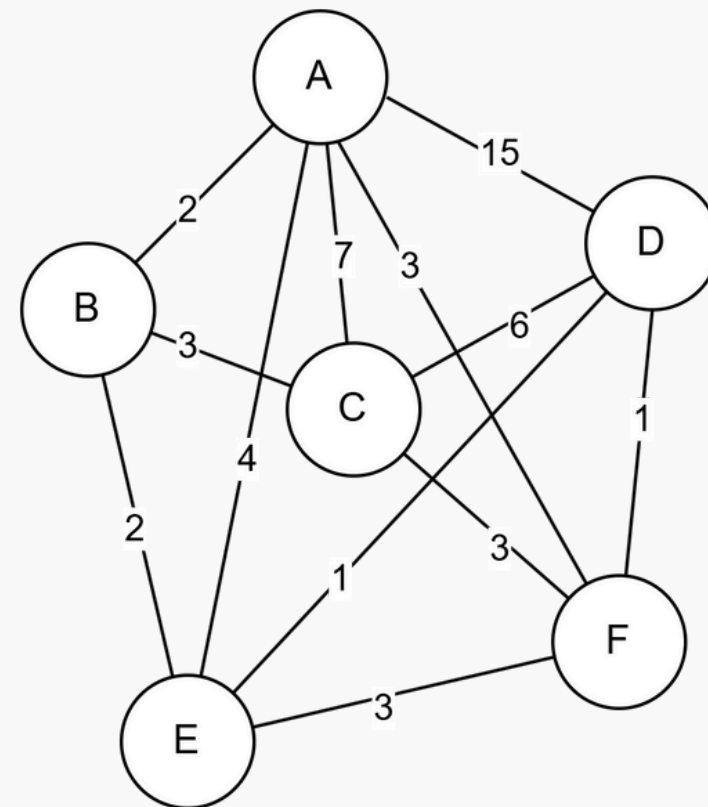
Cambio dado: [10, 10, 10, 2, 1]

Algoritmos de Grafos

01

PRIM

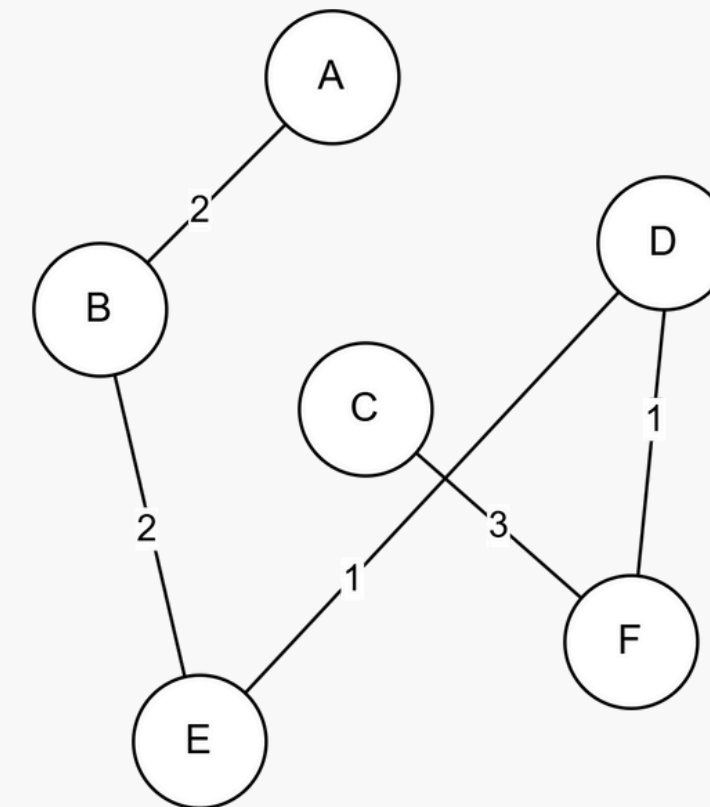
- Árbol de recubrimiento mínimo (MST).
- Grafo No dirigido y ponderado.
- Añade un nodo en cada paso al MST, eligiendo la arista de menor peso disponible.



02

KRUSKAL

- Árbol de recubrimiento mínimo (MST).
- Grafo No dirigido y ponderado.
- Ordena aristas por peso y las agrega al MST.
- Evita formación de ciclos.



Costo MST = 9

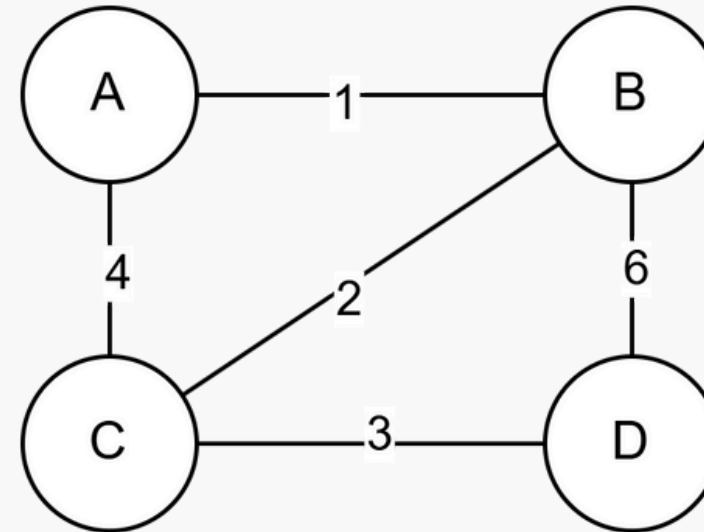
Algoritmos de Grafos

03

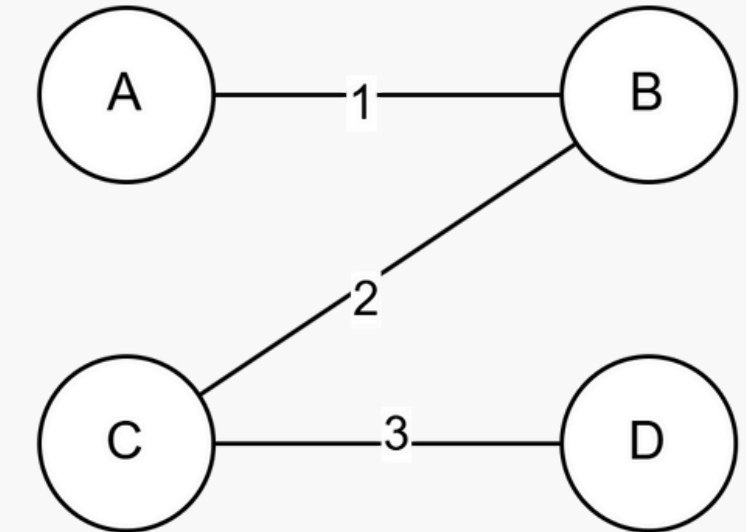
DIJKSTRA

- Camino más corto desde un nodo origen hacia los demás.
- Grafo ponderado.
- Grafos dirigidos o no dirigidos.
- Peso de las aristas no negativos.

Grafo original



Resultado Dijkstra



Las distancias más cortas desde A a los demás nodos son:

A → B: 1

A → C: 3

A → D: 6

Consigna a resolver (PRIM)

Diseño de una red de distribución eléctrica.

Una empresa de energía necesita conectar varias estaciones eléctricas en una región para asegurar que toda la zona esté alimentada de manera eficiente. Las estaciones están ubicadas en diferentes ciudades y los costos de instalación de las líneas eléctricas entre ellas varían según la distancia y el terreno.

Tareas:

Representar el grafo utilizando una lista de adyacencia.

Aplicar el algoritmo de Prim para determinar el Árbol de Recubrimiento Mínimo.

Mostrar el conjunto de conexiones resultante y calcular el costo total.



Resolución ejercicio

Complejidad temporal

$O(n \log n)$

```
public class c6_act3 {  
  
    static class RedElectrica {  
  
        private List<List<int[]>> red;  
        private int numEstaciones;  
  
        RedElectrica(int numEstaciones) {  
            this.numEstaciones = numEstaciones;  
            red = new ArrayList<>(numEstaciones);  
            for (int i = 0; i < numEstaciones; i++) {  
                red.add(new ArrayList<>());  
            }  
        }  
  
        public void agregarLineaElectrica(int estacion1, int estacion2, int costo) {  
            red.get(estacion1).add(new int[]{estacion2, costo});  
            red.get(estacion2).add(new int[]{estacion1, costo});  
        }  
    }  
}
```

↓
Inicializa la red de
distribución y crea la lista de
adyacencia

CP para obtener
nodo con menor
peso

```
public int calcularRedOptima() {  
    boolean[] conectado = new boolean[numEstaciones];  
    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));  
    int costoTotal = 0;  
    List<int[]> conexionesOptimas = new ArrayList<>();  
  
    pq.add(new int[]{0, 0, -1});  
  
    while (!pq.isEmpty()) {  
        int[] actual = pq.poll();  
        int estacion = actual[0];  
        int costo = actual[1];  
        int origen = actual[2];  
  
        if (conectado[estacion]) {  
            continue;  
        }  
        conectado[estacion] = true;  
        costoTotal += costo;  
  
        if (origen != -1) {  
            conexionesOptimas.add(new int[]{origen, estacion, costo});  
        }  
  
        for (int[] vecino : red.get(estacion)) {  
            int estacionVecina = vecino[0];  
            int costoVecino = vecino[1];  
  
            if (!conectado[estacionVecina]) {  
                pq.add(new int[]{estacionVecina, costoVecino, estacion});  
            }  
        }  
    }  
}
```

↓
Aplica el algoritmo PRIM
para calcular el MST

Resolución ejercicio

```
c6_act3.java

System.out.println("Líneas eléctricas en la red óptima:");
for (int[] conexion : conexionesOptimas) {
    System.out.println("Estación " + conexion[0] + " --- Estación "
        + conexion[1] + " (Costo: " + conexion[2] + ")");
}

return costoTotal;
}

public static void main(String[] args) {
    RedElectrica red = new RedElectrica(6);

    red.agregarLineaElectrica(0, 1, 2);
    red.agregarLineaElectrica(0, 2, 7);
    red.agregarLineaElectrica(0, 3, 15);
    red.agregarLineaElectrica(0, 4, 4);
    red.agregarLineaElectrica(0, 5, 3);

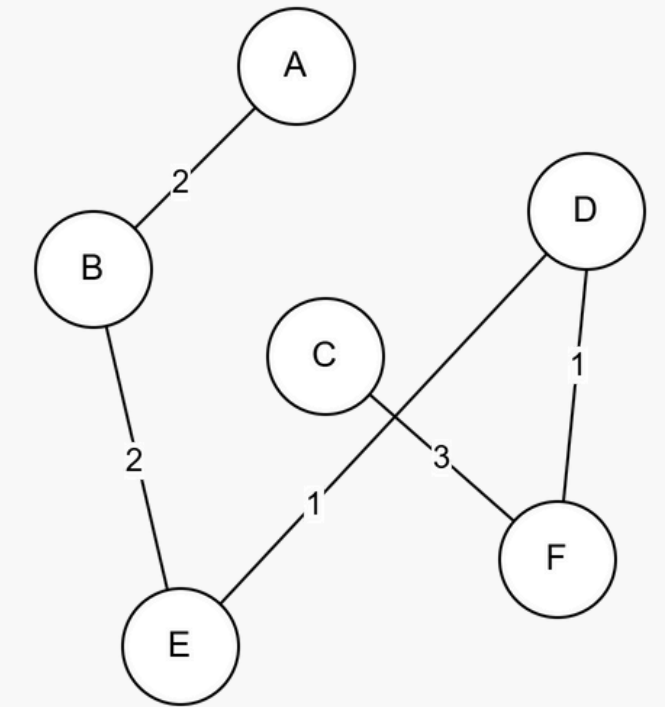
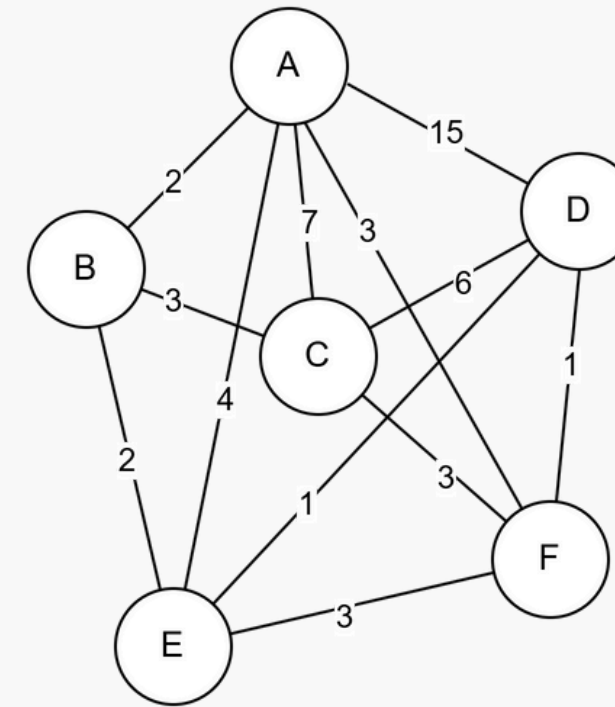
    red.agregarLineaElectrica(1, 2, 3);
    red.agregarLineaElectrica(1, 4, 2);

    red.agregarLineaElectrica(2, 3, 6);
    red.agregarLineaElectrica(2, 5, 3);

    red.agregarLineaElectrica(3, 5, 1);
    red.agregarLineaElectrica(3, 4, 1);

    red.agregarLineaElectrica(4, 5, 3);

    int costoTotalMST = red.calcularRedOptima();
    System.out.println("Costo total mínimo de la red eléctrica: " + costoTotalMST);
}
}
```



Costo MST = 9

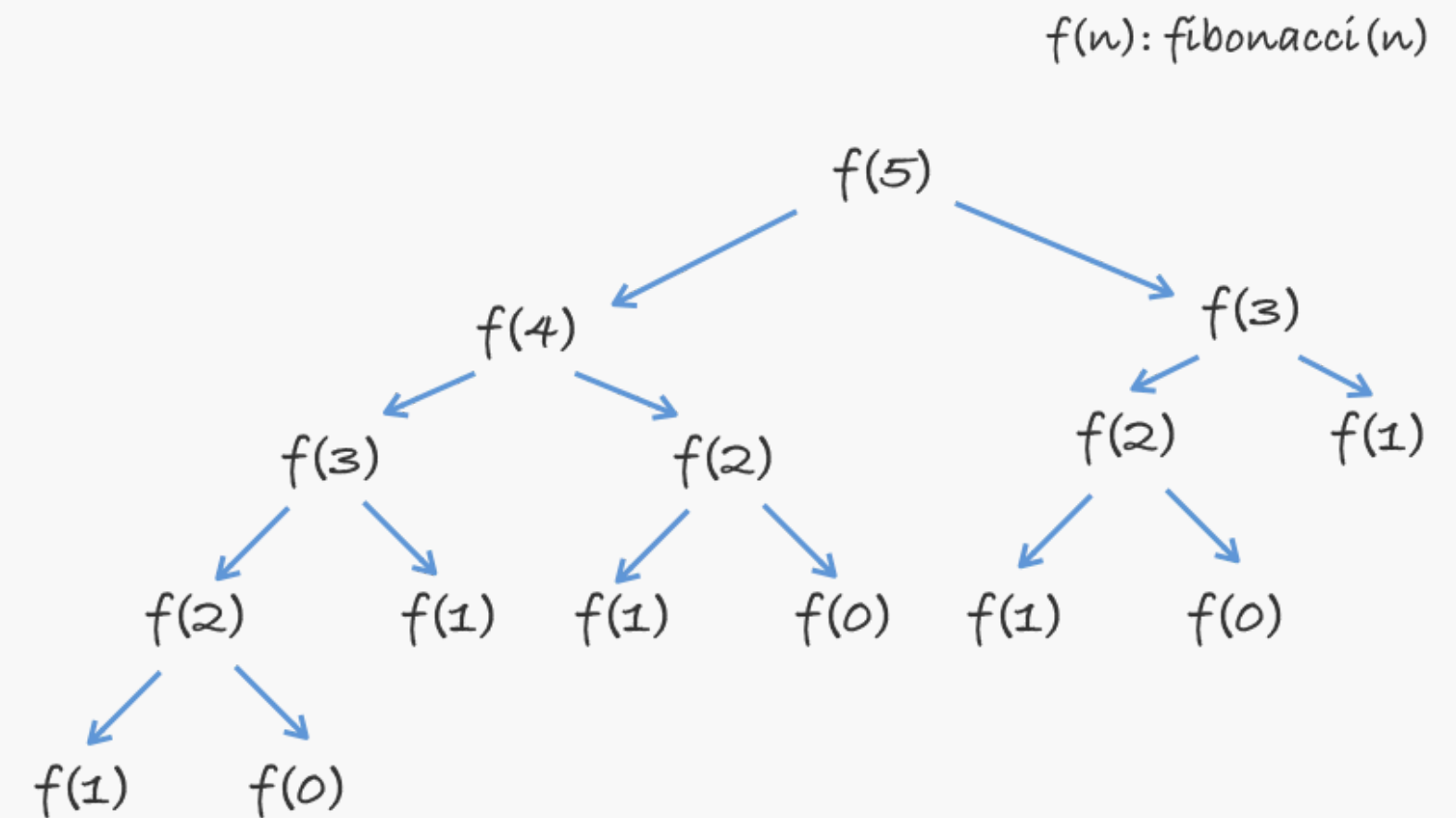
Imprime lista de adyacencia
+ Costo MST

```
Líneas eléctricas en la red óptima:
Estación 0 --- Estación 1 (Costo: 2)
Estación 1 --- Estación 4 (Costo: 2)
Estación 4 --- Estación 3 (Costo: 1)
Estación 3 --- Estación 5 (Costo: 1)
Estación 5 --- Estación 2 (Costo: 3)
Costo total mínimo de la red eléctrica: 9
```

Programación Dinámica

Dividir problemas complejos en subproblemas más simples

Es una técnica de diseño de algoritmos que se utiliza para resolver problemas complejos, dividiéndolos en subproblemas más simples y almacenando los resultados para evitar cálculos redundantes.

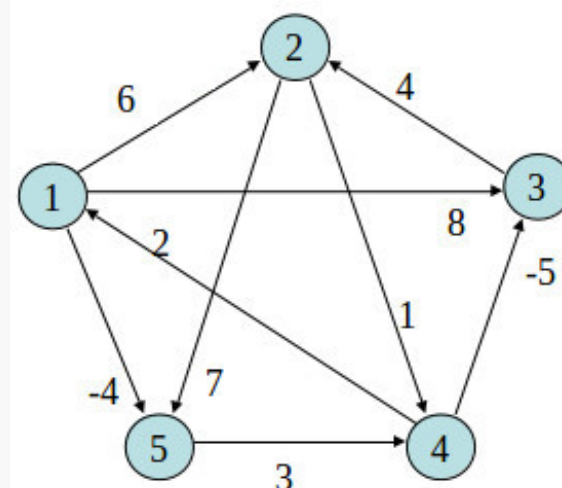


Algoritmo de Floyd-Warshall

Camino más corto
entre TODOS los
nodos

Grafos ponderados
dirigidos o no
dirigidos

Matriz de
adyacencia



	1	2	3	4	5
1	0	6	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	∞	-5	0	∞
5	∞	∞	∞	3	0

Consigna a resolver (PD)

Para un grafo ponderado, se requiere determinar la ruta más corta entre cada par de vértices del conjunto, considerando los pesos de las aristas.



Pasos a seguir

Se obtendra como resultado una matriz con los pesos de los caminos mas cortos para conectar cada par de vertices

1

Inicializar la matriz $\text{graph}[i][i]$, con $i > 0$

2

Aqui se encuentran los caminos con los pesos iniciales

3

Se llama al algoritmo

4

Se inicializa otra matriz $\text{dist}[v][v]$ donde $v = i$

5

Se itera para vertice K

6

se evalua que el valor nuevo sea menor al anterior

7

Si lo es se reemplaza el valor

8

Se imprime la matriz resultado (dist)

Resolución ejercicio

Inicialización de la matriz resultado

Iteración para K vertices

Complejidad temporal de tipo cubica, **$O(n^3)$** con n la cantidad de vertices

```
c8_act1.java

package clase8;

import java.util.Arrays;

public class Actividad1 {
    static final int INF = 99999;

    public static void floydWarshall(int graph[][]) {
        int V = graph.length;
        int dist[][] = new int[V][V];

        for (int i = 0; i < V; i++) {
            dist[i] = Arrays.copyOf(graph[i], V);
        }

        for (int k = 0; k < V; k++) {
            for (int i = 0; i < V; i++) {
                for (int j = 0; j < V; j++) {
                    if (dist[i][k] != INF && dist[k][j] != INF && dist[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                    }
                }
            }
        }
        printSolution(dist);
    }
}
```

Resolución ejercicio

c8_act1.java

```
public static void printSolution(int dist[][]) {
    System.out.println("Matriz de distancias más cortas entre todos los pares de nodos:");
    for (int i = 0; i < dist.length; i++) {
        for (int j = 0; j < dist.length; j++) {
            if (dist[i][j] == INF)
                System.out.print("INF ");
            else
                System.out.print(dist[i][j] + " ");
        }
        System.out.println();
    }
}

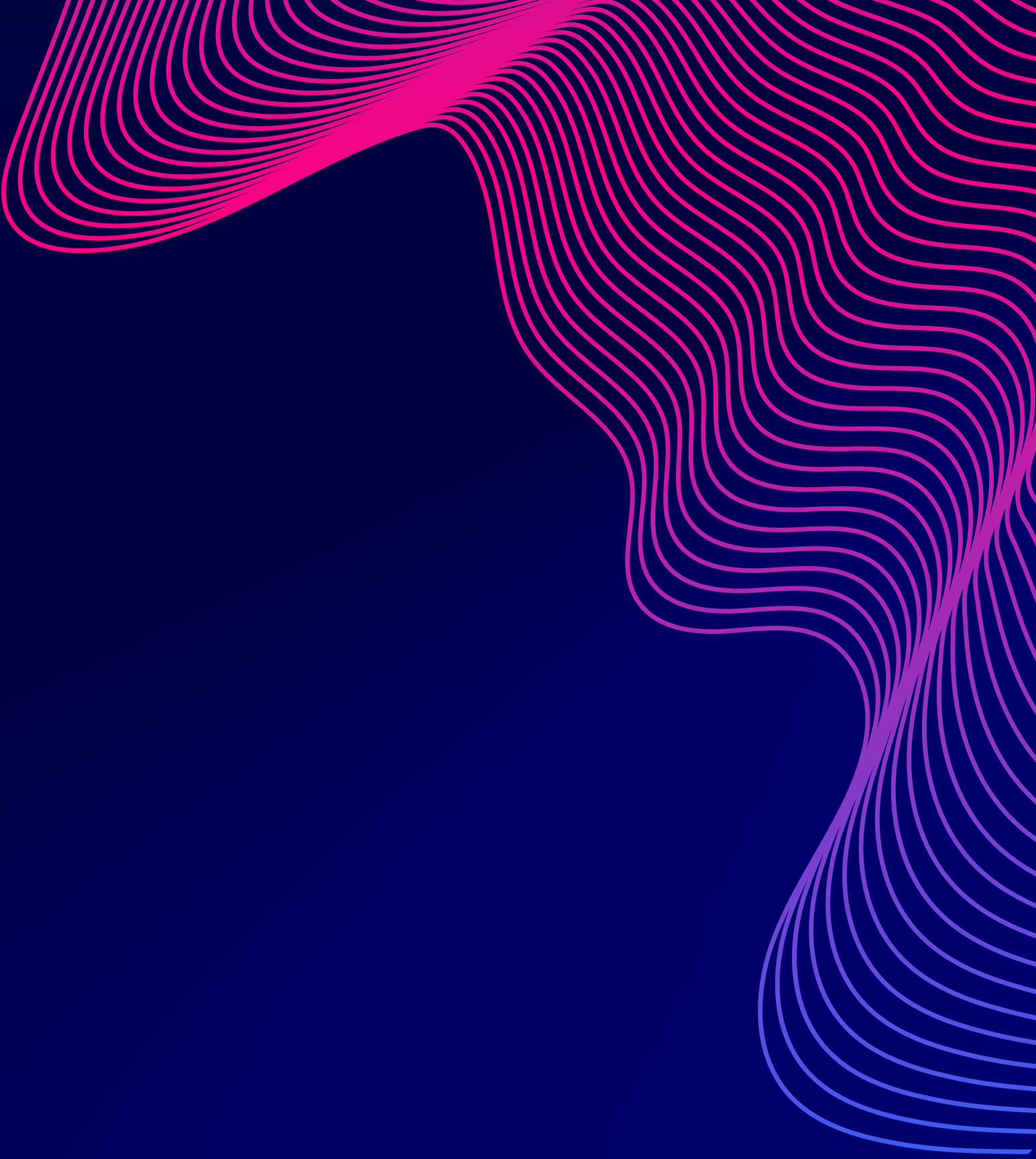
public static void main(String[] args) {
    int graph[][] = {
        {0, 3, INF, 5},
        {2, 0, INF, 4},
        {INF, 1, 0, INF},
        {INF, INF, 2, 0}
    };

    floydWarshall(graph);
}
```

Impresion de la matriz

Complejidad de tipo
cuadratica del tipo **$O(n^2)$**

Inicializacion de la matriz

A decorative graphic in the top right corner consisting of numerous thin, wavy, concentric lines that create a sense of depth and movement, transitioning from a bright pink color at the top to a lighter purple at the bottom.

¡Muchas gracias!

Grupo 5

SÁNCHEZ AGÜERO, FRANCO

YAÑEZ, AGUSTÍN EMANUEL

CARRIZO, FACUNDO MARTIN